

Unit 2: Supervised Machine Learning Algorithms

Dr. Puneet Bawa

Introduction to Machine Learning

Academic Session 2026–27

By the end of this lecture, students should be able to:

- distinguish regression from classification problems;
- explain Linear Regression, Logistic Regression, Decision Tree, Random Forest, Naive Bayes and KNN;
- calculate Gini impurity, entropy and information gain;
- explain bootstrap sampling and Laplace smoothing;
- select an appropriate algorithm for a given problem.

Regression and classification

Task	Question	Output
Regression	How many marks will a student score?	A continuous value such as 76.5
Classification	Will the student pass or fail?	A class such as Pass or Fail

Core distinction

Regression predicts a quantity. Classification predicts a category or its probability.

Linear Regression: purpose

Linear Regression predicts a continuous numerical target by fitting a linear relationship between inputs and output.

Examples

- marks from study hours;
- house price from area;
- sales from advertising expenditure.

Basic assumption

- the average target changes linearly with the input;
- observations contain random error around the line.

Simple Linear Regression equation

$$\hat{y} = b_0 + b_1x$$

- x : input feature, such as study hours;
- $\hat{y}$: predicted value, such as predicted marks;
- b_0 : intercept, or predicted value when $x = 0$;
- b_1 : slope, or predicted change in y for a one-unit increase in x .

Interpretation

If $\widehat{Marks} = 25 + 8(Study\ Hours)$, one additional study hour increases the predicted marks by 8. This describes association, not necessarily causation.

Linear Regression numerical example

Given:

$$\widehat{Marks} = 25 + 8(Study\ Hours)$$

For a student studying 5 hours:

$$\widehat{Marks} = 25 + 8(5) = 65$$

If the actual marks are 68, the residual is:

$$e = y - \hat{y} = 68 - 65 = 3$$

Meaning

The model under-predicted this student's marks by 3.

How Linear Regression learns

The model selects coefficients that minimize the sum of squared errors:

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- Squaring prevents positive and negative errors from cancelling.
- Large errors receive a stronger penalty.
- The fitted line need not pass through every observation.

Common evaluation measures:

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|, \quad RMSE = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$$

Multiple Linear Regression

When several features predict the target:

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \cdots + b_px_p$$

Example:

$$\widehat{Marks} = 10 + 4(Study\ Hours) + 0.5(Attendance)$$

For 5 study hours and 80% attendance:

$$\widehat{Marks} = 10 + 4(5) + 0.5(80) = 70$$

Caution

Highly correlated predictors can make individual coefficients unstable. Outliers may also pull the fitted line.

Logistic Regression: purpose

Logistic Regression predicts the probability of a class, usually for binary classification.

- Pass or Fail
- Spam or Not Spam
- Disease or No Disease
- Fraud or Legitimate transaction

Why not use Linear Regression?

A probability must remain between 0 and 1. A straight line can produce values below 0 or above 1.

From a linear score to probability

First calculate a linear score:

$$z = b_0 + b_1x_1 + \cdots + b_px_p$$

Then apply the sigmoid function:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

z	-3	-2	-1	0	1	3
p	.047	.119	.269	.500	.731	.953

The sigmoid maps every real-valued score to the interval $(0, 1)$.

Logistic Regression numerical example

Suppose:

$$z = -4 + 1.2(\textit{Study Hours})$$

For 5 study hours:

$$z = -4 + 1.2(5) = 2$$

$$p = \frac{1}{1 + e^{-2}} \approx 0.881$$

Using a threshold of 0.5:

$$0.881 \geq 0.5 \quad \Rightarrow \quad \boxed{\textit{Pass}}$$

The threshold can be changed when false positives and false negatives have different costs.

Log-odds and learning

Logistic Regression models the log-odds linearly:

$$\log \left(\frac{p}{1-p} \right) = b_0 + b_1 x_1 + \dots + b_p x_p$$

It commonly minimizes binary cross-entropy:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

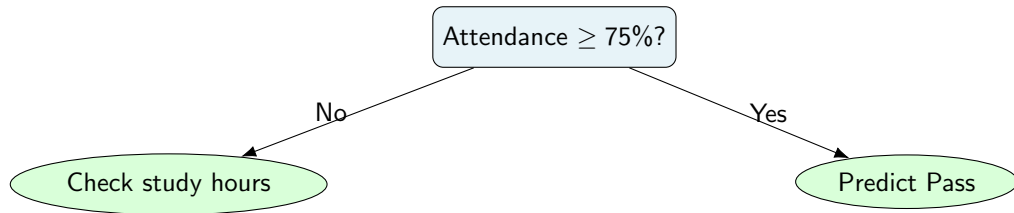
Intuition

A confident but wrong prediction receives a large penalty.

Evaluate with: confusion matrix, precision, recall, F1-score, ROC-AUC and log loss.

Decision Tree: basic idea

A Decision Tree predicts by asking a sequence of feature-based questions.



- Root node: first question
- Internal node: later question
- Branch: outcome of a question
- Leaf: final prediction

How a tree chooses a split

For classification, a tree searches for a split that creates purer child nodes.

- A pure node contains observations from one class.
- An impure node contains a mixture of classes.
- Common criteria are Gini impurity and entropy.

Criterion	Preferred split
Gini impurity	Lowest weighted child Gini
Entropy	Highest information gain
Regression tree	Lowest weighted squared error

Gini impurity

For K classes:

$$G = 1 - \sum_{k=1}^K p_k^2$$

For 6 Pass and 4 Fail students:

$$G_{parent} = 1 - \left(\frac{6}{10}\right)^2 - \left(\frac{4}{10}\right)^2 = 1 - .36 - .16 = \boxed{.48}$$

- $G = 0$: pure node
- For two classes, maximum $G = 0.5$ at a 50–50 distribution

Gini split: child nodes

Candidate split: **Attendance** $\geq 75\%$

Left child: 1 Pass, 3 Fail

$$G_L = 1 - \left(\frac{1}{4}\right)^2 - \left(\frac{3}{4}\right)^2 = \boxed{.375}$$

Right child: 5 Pass, 1 Fail

$$G_R = 1 - \left(\frac{5}{6}\right)^2 - \left(\frac{1}{6}\right)^2 = \boxed{.278}$$

Weighted Gini:

$$G_{split} = \frac{4}{10}(.375) + \frac{6}{10}(.278) = \boxed{.317}$$

Gini gain and split selection

$$\text{Gini Gain} = G_{\text{parent}} - G_{\text{split}} = .48 - .317 = \boxed{.163}$$

Candidate	Weighted Gini	Gini gain
Attendance $\geq 75\%$	.317	.163
Study hours ≥ 3	.400	.080

The tree chooses **Attendance** $\geq 75\%$ because it has lower weighted impurity and higher impurity reduction.

Entropy

Entropy measures uncertainty:

$$H(S) = - \sum_{k=1}^K p_k \log_2 p_k$$

For 6 Pass and 4 Fail students:

$$H(S) = - [.6 \log_2(.6) + .4 \log_2(.4)]$$

$$H(S) = - [(.6)(-.737) + (.4)(-1.322)] = \boxed{.971}$$

For binary classification, $H = 0$ means pure and $H = 1$ represents maximum uncertainty.

Entropy split calculation

For the Attendance split:

$$H_L = - \left[\frac{1}{4} \log_2 \frac{1}{4} + \frac{3}{4} \log_2 \frac{3}{4} \right] = .811$$

$$H_R = - \left[\frac{5}{6} \log_2 \frac{5}{6} + \frac{1}{6} \log_2 \frac{1}{6} \right] = .650$$

Weighted entropy:

$$H_{split} = \frac{4}{10}(.811) + \frac{6}{10}(.650) = \boxed{.714}$$

Information gain:

$$IG = H_{parent} - H_{split} = .971 - .714 = \boxed{.257}$$

Comparing entropy-based splits

Candidate	Weighted entropy	Information gain
Attendance $\geq 75\%$	.714	.257
Study hours ≥ 3	.847	.124

Selection rule

Choose the split with the highest information gain. It creates the largest reduction in uncertainty.

Tree overfitting and control

A deep tree may memorize noise in the training data.

- `max_depth`: limits the number of levels
- `min_samples_split`: minimum records needed to split
- `min_samples_leaf`: minimum records in a leaf
- `pruning`: removes branches that add little predictive value

Trade-off

A very shallow tree may underfit. A very deep tree may overfit. Validation helps find an appropriate complexity.

Random Forest: main idea

A Random Forest combines many randomized Decision Trees.

- ① Draw a bootstrap sample for each tree.
- ② At each split, consider a random subset of features.
- ③ Grow many trees.
- ④ Aggregate their predictions.
 - Classification: average class probabilities and choose the largest.
 - Regression: average numerical predictions.

Bootstrap sampling

A bootstrap sample draws observations **with replacement** and usually has the same size as the original dataset.

$$D = [A, B, C, D, E]$$

Possible samples:

$$D_1 = [B, D, B, A, E]$$

$$D_2 = [C, A, E, C, D]$$

In D_1 , B appears twice and C does not appear.

With replacement

After an observation is selected, it returns to the sampling pool and can be selected again.

Bagging and out-of-bag records

Bagging = Bootstrap sampling + Aggregation

An observation excluded from a tree's bootstrap sample is an **out-of-bag (OOB)** observation for that tree.

Example

If $D = [A, B, C, D, E]$ and $D_1 = [B, D, B, A, E]$, observation C is OOB for Tree 1.

For a large dataset, a bootstrap sample contains approximately 63.2% unique original observations. Approximately 36.8% remain OOB for a given tree.

Random Forest aggregation

Classification example

Tree	1	2	3	4	5
Prediction	Pass	Pass	Fail	Pass	Fail

Pass receives 3 of 5 illustrative votes, so the final class is Pass. **Regression example**

$$\hat{y} = \frac{65 + 70 + 68 + 72 + 75}{5} = \boxed{70}$$

Random Forest strengths and limitations

Strengths

- captures nonlinear patterns;
- reduces variance through averaging;
- performs well on many tabular datasets;
- usually needs no feature scaling.

Limitations

- less interpretable than one small tree;
- requires more memory and computation;
- feature importance does not prove causation.

Naive Bayes: main idea

Naive Bayes predicts the class with the highest posterior probability.

$$P(C | X) = \frac{P(X | C)P(C)}{P(X)}$$

For comparison between classes:

$$P(C | X) \propto P(C)P(X | C)$$

Under the conditional independence assumption:

$$P(C | x_1, \dots, x_p) \propto P(C) \prod_{j=1}^p P(x_j | C)$$

It is called “naive” because it assumes features are conditionally independent given the class.

Naive Bayes terms

Term	Meaning
$P(C)$	Prior: class probability before observing features
$P(X C)$	Likelihood: probability of features within a class
$P(C X)$	Posterior: updated class probability after observing features
$P(X)$	Evidence: overall probability of the observed features

Naive Bayes numerical example: data

Ten students include 6 Pass and 4 Fail:

$$P(\text{Pass}) = .6, \quad P(\text{Fail}) = .4$$

For a student with high attendance and a submitted assignment:

Conditional probability	Pass	Fail
High attendance	5/6	1/4
Assignment submitted	4/6	1/4

Naive Bayes numerical example: calculation

Pass score:

$$.6 \times \frac{5}{6} \times \frac{4}{6} = \boxed{.333}$$

Fail score:

$$.4 \times \frac{1}{4} \times \frac{1}{4} = \boxed{.025}$$

Normalised posterior for Pass:

$$P(\text{Pass} \mid X) = \frac{.333}{.333 + .025} \approx \boxed{.930}$$

The model predicts **Pass** because the Pass posterior is larger.

Types of Naive Bayes

Variant	Suitable data
Gaussian Naive Bayes	Continuous measurements such as age or marks
Multinomial Naive Bayes	Counts such as word frequencies
Bernoulli Naive Bayes	Binary features such as present or absent
Categorical Naive Bayes	Discrete categories

The zero-frequency problem

Suppose no Not-Spam email in the training set contains the word “Offer”:

$$P(\text{Offer} \mid \text{NotSpam}) = \frac{0}{3} = 0$$

Naive Bayes multiplies probabilities. One zero makes the complete class score zero:

$$P(\text{NotSpam} \mid X) \propto P(\text{NotSpam}) \times 0 \times \dots = 0$$

Problem

The model completely eliminates the class even when other features support it.

Laplace smoothing

Laplace smoothing adds 1 to every possible category count:

$$P(x = v \mid C) = \frac{\text{Count}(x = v, C) + 1}{\text{Count}(C) + K}$$

K is the number of possible feature values. For $\text{Offer} = \{\text{Present}, \text{Absent}\}$, $K = 2$:

$$P(\text{Offer} = \text{Present} \mid \text{NotSpam}) = \frac{0 + 1}{3 + 2} = \boxed{.20}$$

$$P(\text{Offer} = \text{Absent} \mid \text{NotSpam}) = \frac{3 + 1}{3 + 2} = \boxed{.80}$$

The two probabilities still sum to 1.

General additive smoothing

$$P(x = v \mid C) = \frac{\text{Count}(x = v, C) + \alpha}{\text{Count}(C) + \alpha K}$$

- $\alpha = 1$: Laplace smoothing
- $0 < \alpha < 1$: Lidstone smoothing
- $\alpha = 0$: no smoothing

For Multinomial Naive Bayes:

$$P(\text{word} \mid C) = \frac{\text{Count}(\text{word}, C) + 1}{\text{total word count in } C + |V|}$$

where $|V|$ is the vocabulary size.

KNN: main idea

K-Nearest Neighbours predicts a new observation from its K closest training observations.

- 1 Choose K .
- 2 Calculate distances to all training observations.
- 3 Select the K nearest observations.
- 4 Vote for classification or average for regression.

KNN is a **lazy learner**: it stores training data and performs most computation during prediction.

Euclidean distance

For two features:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

For p features:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^p (x_j - y_j)^2}$$

Other measures include Manhattan, Minkowski and Hamming distance.

Neighbour assumption

Observations that are close in feature space are expected to have similar outputs.

KNN numerical example: dataset

Predict the class of $X = (4, 60)$.

Student	Study hours	Assignment score	Class
A	2	40	Fail
B	3	50	Fail
C	5	70	Pass
D	6	80	Pass
E	7	85	Pass

KNN numerical example: distances

$$d(X, A) = \sqrt{(4 - 2)^2 + (60 - 40)^2} = 20.10$$

$$d(X, B) = \sqrt{(4 - 3)^2 + (60 - 50)^2} = 10.05$$

$$d(X, C) = \sqrt{(4 - 5)^2 + (60 - 70)^2} = 10.05$$

$$d(X, D) = \sqrt{(4 - 6)^2 + (60 - 80)^2} = 20.10$$

$$d(X, E) = \sqrt{(4 - 7)^2 + (60 - 85)^2} = 25.18$$

For $K = 3$, the selected neighbours are B, C and A with classes Fail, Pass and Fail.

Prediction = Fail

Choosing K

Small K

- sensitive to noise;
- complex boundary;
- may overfit.

Large K

- smoother boundary;
- may hide local patterns;
- may underfit or favour the majority class.

Use validation or cross-validation to select K . An odd value may reduce ties in binary classification.

Why feature scaling matters in KNN

Distance can be dominated by a feature with a larger numerical range.

- Study hours: 1 to 10
- Annual income: 200,000 to 2,000,000

Income would dominate an unscaled Euclidean distance. **Standardisation**

$$z = \frac{x - \mu}{\sigma}$$

Min-max normalisation

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Fit the scaler on training data only.

Weighted KNN

Ordinary KNN gives each selected neighbour an equal vote. Weighted KNN gives closer neighbours more influence.

$$w_i = \frac{1}{d_i + \epsilon}$$

Classification selects the class with the largest total neighbour weight.

Use case

Weighted voting helps when the closest observation should influence the decision more than a neighbour near the edge of the selected region.

Algorithm comparison

Algorithm	Task	Main idea	Scaling	Interpretability
Linear Regression	Regression	Fits a linear equation	Often useful	High
Logistic Regression	Classification	Models class probability	Often useful	High
Decision Tree	Both	Learns sequential rules	Usually unnecessary	High when small
Random Forest	Both	Averages randomized trees	Usually unnecessary	Moderate to low
Naive Bayes	Classification	Compares posterior probabilities	Usually unnecessary	Moderate
KNN	Both	Uses nearest observations	Usually essential	Intuitive, local

Which model fits the problem?

Problem	Suitable starting model
Predict marks from study hours	Linear Regression
Predict Pass or Fail with a probability	Logistic Regression
Explain decisions as simple rules	Decision Tree
Model nonlinear tabular patterns robustly	Random Forest
Classify documents using word counts	Multinomial Naive Bayes
Predict from nearby similar observations	KNN

No algorithm wins for every dataset. Compare models using unseen data and suitable evaluation metrics.

Common misconceptions

- Logistic Regression is mainly a classification algorithm.
- Decision Trees and Random Forests can perform classification and regression.
- Random Forest is a collection of trees, not one large tree.
- Naive Bayes assumes conditional independence, not unconditional independence.
- KNN usually requires feature scaling.
- Feature importance and model coefficients do not automatically prove causation.
- High training accuracy does not guarantee good performance on new data.

Quick revision questions

- ① Why does Logistic Regression use the sigmoid function?
- ② What does a Gini impurity of zero mean?
- ③ How does information gain select a Decision Tree split?
- ④ Why can one record occur several times in a bootstrap sample?
- ⑤ What problem does Laplace smoothing solve?
- ⑥ Why must KNN features usually be scaled?

Summary

- Linear Regression estimates continuous values using a linear equation.
- Logistic Regression converts a linear score into class probability.
- Decision Trees choose splits that reduce impurity.
- Random Forests combine bootstrapped, feature-randomized trees.
- Naive Bayes applies Bayes' theorem under a conditional-independence assumption.
- Laplace smoothing prevents zero conditional probabilities.
- KNN predicts from nearby observations and depends strongly on distance and scaling.

Thank You

Questions and Discussion